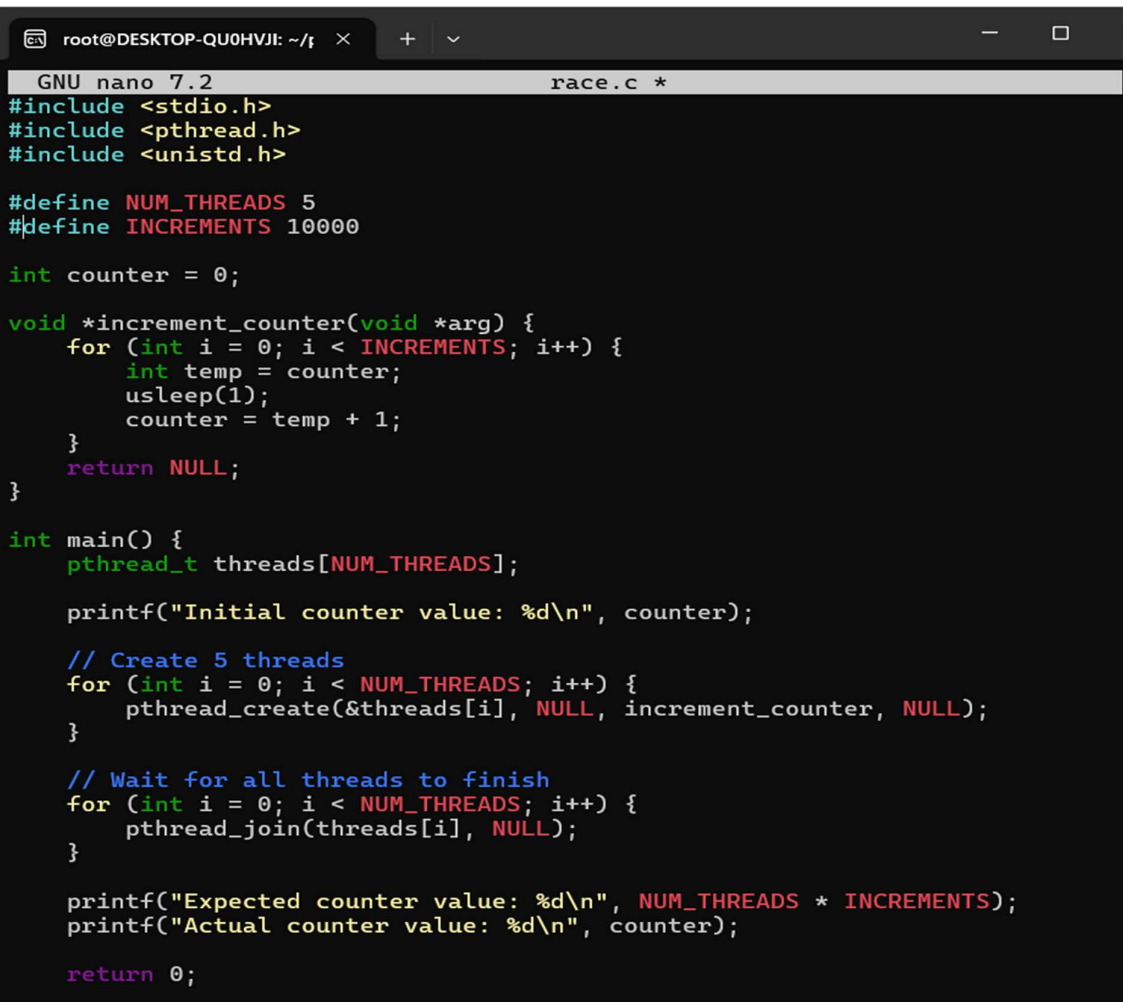


PRACTICAL WEEK - 11

TASK1: Develop a multithreaded counter application using POSIX threads (pthread_create, pthread_join). Demonstrate race conditions by allowing multiple threads to update a shared variable concurrently.

```
root@DESKTOP-QU0HVJI:~/practical/practical-11# nano race.c
root@DESKTOP-QU0HVJI:~/practical/practical-11# gcc race.c -o race -pthread
root@DESKTOP-QU0HVJI:~/practical/practical-11# ./race
Initial counter value: 0
Expected counter value: 50000
Actual counter value: 10002
root@DESKTOP-QU0HVJI:~/practical/practical-11# |
```



```
GNU nano 7.2 race.c *
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

#define NUM_THREADS 5
#define INCREMENTS 10000

int counter = 0;

void *increment_counter(void *arg) {
    for (int i = 0; i < INCREMENTS; i++) {
        int temp = counter;
        usleep(1);
        counter = temp + 1;
    }
    return NULL;
}

int main() {
    pthread_t threads[NUM_THREADS];

    printf("Initial counter value: %d\n", counter);

    // Create 5 threads
    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_create(&threads[i], NULL, increment_counter, NULL);
    }

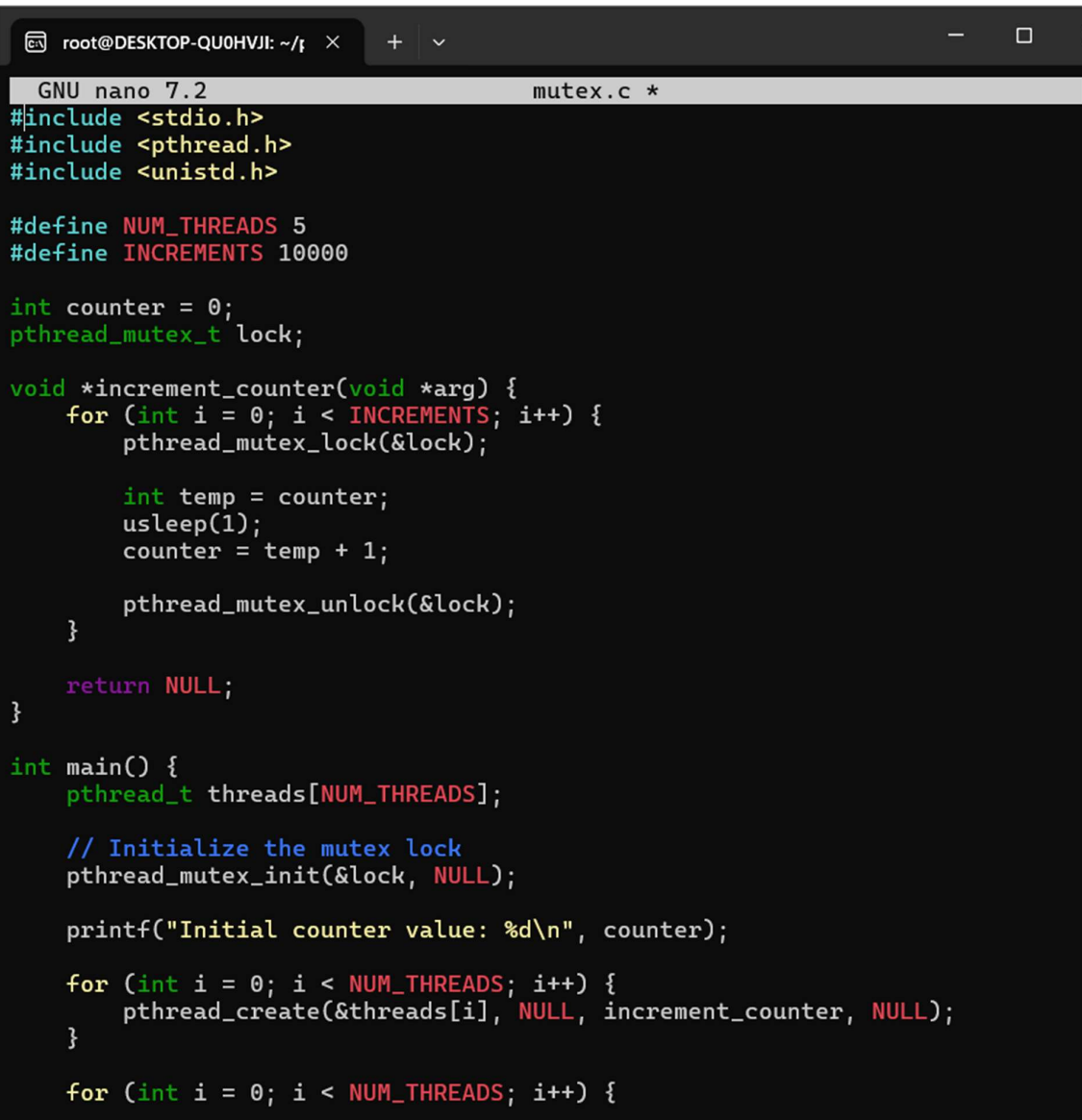
    // Wait for all threads to finish
    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_join(threads[i], NULL);
    }

    printf("Expected counter value: %d\n", NUM_THREADS * INCREMENTS);
    printf("Actual counter value: %d\n", counter);

    return 0;
}
```

TASK2: Modify the previous program using mutex locks (`pthread_mutex_lock`, `pthread_mutex_unlock`) and compare the outputs with the race-condition version. Analyze the impact of synchronization.

```
root@DESKTOP-QU0HVJI:~/practical/practical-11# nano mutex.c
root@DESKTOP-QU0HVJI:~/practical/practical-11# gcc mutex.c -o mutex
root@DESKTOP-QU0HVJI:~/practical/practical-11# ./mutex
Initial counter value: 0
Expected counter value: 50000
Actual counter value: 50000
root@DESKTOP-QU0HVJI:~/practical/practical-11# |
```



```
GNU nano 7.2 mutex.c *
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

#define NUM_THREADS 5
#define INCREMENTS 10000

int counter = 0;
pthread_mutex_t lock;

void *increment_counter(void *arg) {
    for (int i = 0; i < INCREMENTS; i++) {
        pthread_mutex_lock(&lock);

        int temp = counter;
        usleep(1);
        counter = temp + 1;

        pthread_mutex_unlock(&lock);
    }
    return NULL;
}

int main() {
    pthread_t threads[NUM_THREADS];

    // Initialize the mutex lock
    pthread_mutex_init(&lock, NULL);

    printf("Initial counter value: %d\n", counter);

    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_create(&threads[i], NULL, increment_counter, NULL);
    }

    for (int i = 0; i < NUM_THREADS; i++) {
```